GLOBAL SOLUTION 2026-2 · CITIZENTECH

PaperTrail
Modelo de Database

Preparação de documentos e etapas antes de um atendimento público.

PT-DB-BASE-2 · 09/11/2026 · Base inicial para revisão e testes

Leitura rápida

Oracle: 10 tabelas, com catálogo Java e preparativos .NET separados por propriedade. Dataset fictício: 3 serviços, 8 requisitos, 7 preparativos e 14 itens. Um script autônomo, `papertrail.sql`, reúne modelo, carga e testes da base.

Apenas obrigatórios decidem progresso. Vazio e somente opcionais resultam em 0% e não ficam prontos. O modelo guarda snapshots pessoais e prevê auditoria e idempotência.

Problema e alcance

O PaperTrail ajuda o cidadão a consultar requisitos e organizar um checklist antes de procurar um serviço público. O checklist não garante atendimento, aprovação ou elegibilidade. O MVP prevê três serviços de uma cidade, ainda por selecionar. Não recebe documentos pessoais ou pagamentos.

Este PDF reúne o modelo inteiro e suas regras. A base possui DDL, dados e testes escritos. Packages, gatilhos, blocos demonstrativos, MongoDB e relatórios continuam pendentes; isso não constitui a entrega final da disciplina.

Ambiente inicial: Oracle da FIAP

Usar a conta acadêmica existente, com os objetos do PaperTrail separados por prefixo `PT_`. Endereço registrado no histórico do ClyvoCare: `oracle.fiap.com.br:1521`, serviço `ORCL`. Conexão e instalação da base confirmadas em 09/11/2026 no Oracle Database 19c Enterprise Edition. A conta acadêmica é a mesma do ClyvoCare; a criação ficou restrita aos nomes do PaperTrail.

Não há usuário ou senha neste documento ou no SQL. Não reutilizar entidades, migrations ou objetos do ClyvoCare. Na mesma conta podem existir tabelas de outros trabalhos; elas não devem ser removidas.

Executar o script único

1. Conectar no SQL Developer, SQLcl ou SQL*Plus com a conta FIAP do próprio integrante.
2. Conferir a conexão e abrir `docs/database/papertrail.sql`.
3. No SQL Developer, executar como script (F5). No SQLcl/SQL*Plus: `@docs/database/papertrail.sql`, a partir da raiz do repositório.
4. Guardar a saída real. A fase de testes deve terminar com a mensagem de sucesso. Uma mensagem de erro interrompe a execução.

> Antes do DDL, o script consulta os nomes existentes e interrompe se encontrar objetos ou constraints reservados ao PaperTrail. Não contém DROP, TRUNCATE ou limpeza automática. Não reexecutar sobre uma instalação já existente.

Ordem: verificação de nomes → tabelas Java → tabelas .NET → dados fictícios → COMMIT da carga → testes com rollback das alterações de teste. Oracle confirma DDL implicitamente; uma falha posterior pode deixar objetos criados. Não executar com transação de trabalho pendente. Se houver erro, revisar os objetos identificados, sem apagar o schema ou objetos de outros projetos.

DDL usa Oracle 19c como referência de sintaxe. A versão 19c, as permissões e a quota necessárias à instalação desta base foram verificadas pela execução real na FIAP. Os testes escritos cobrem contagens, constraints, identity, unicidade de idempotência e rollback; não comprovam concorrência ou integração HTTP.

Arquivos e manutenção

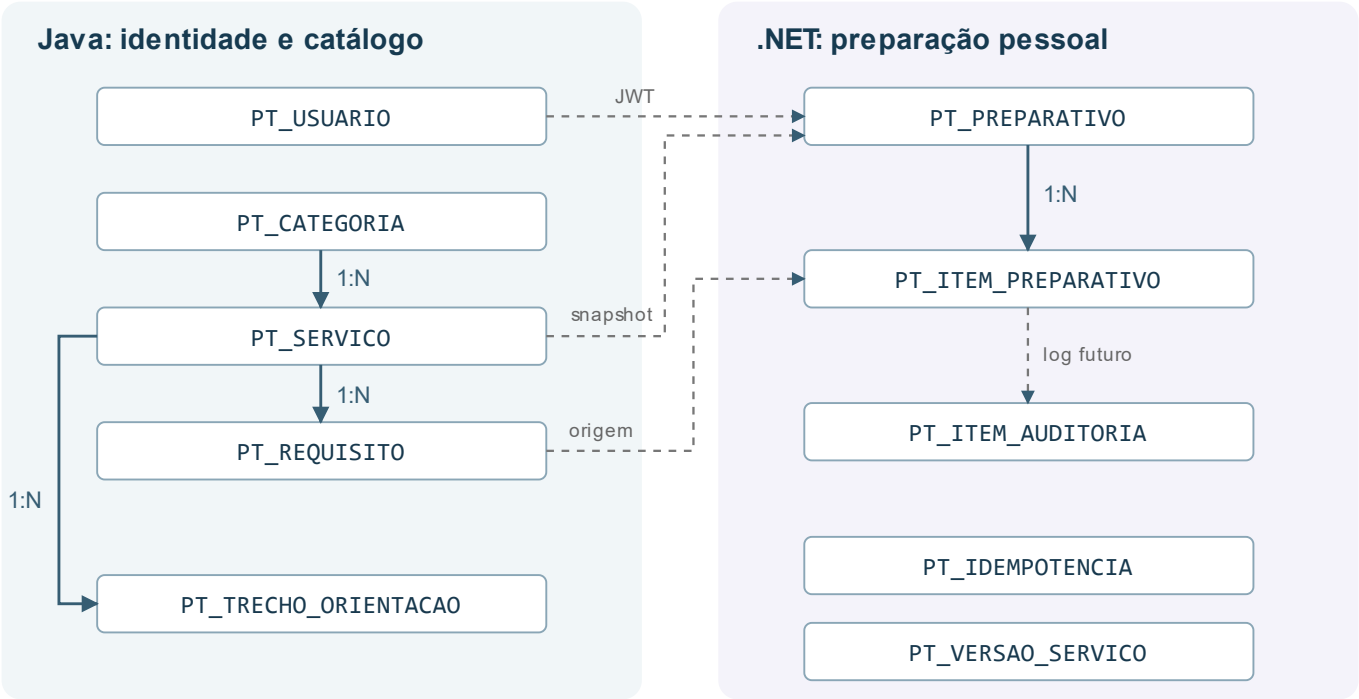
A documentação Java à a origem do material. `docs/database/` contém somente este PDF e o script `papertrail.sql`. Quando o .NET existir, copiar os mesmos artefatos, sem editar as cópias independentemente.

Os fontes de manutenção ficam em `tools/database/`: `documentacao.html`, `fixtures.json` e `generate-fixtures.mjs`. O HTML à o fonte editável deste PDF; para regenerar, abrir no navegador e imprimir em PDF A4, com gráficos de fundo e sem cabeçalhos automáticos.

```
node tools/database/generate-fixtures.mjs --check
node tools/database/generate-fixtures.mjs
```

O primeiro comando valida os dados e confere o trecho gerado dentro do SQL. O segundo regenera somente a seção de fixtures, preservando DDL, testes e comentários. Nenhum deles conecta a banco. Não há dependência npm adicional.

Contextos e relacionamentos



Setas contínuas são FKs. Pontilhadas representam contratos lógicos, não joins obrigatórios nem integrações implementadas. `PT_IDEMPOTENCIA` guarda o resultado da operação sem depender da existência posterior do preparativo.

O catálogo Java não consulta tabelas privadas do .NET. O .NET recebe dados do catálogo por HTTP e verifica identidade pelo JWT. Nos testes iniciais, os objetos PT_ usarão a mesma conta/schema FIAP do ClyvoCare, sem modificar os objetos existentes. Os contextos continuam separados por propriedade, sem FK entre eles.

Convenções físicas

Convenção	Definição
IDs e versões	`NUMBER(18)`, inteiros positivos, compatíveis com `long`/`Long`. Limite físico: 18 dígitos.
Geração de IDs	`GENERATED BY DEFAULT ON NULL AS IDENTITY`, início 1000; fixtures usam IDs explícitos menores.
Booleanos Oracle	`NUMBER(1)` com `CHECK ... IN (0,1)` e `NOT NULL`; HTTP e Mongo usam booleanos.
Textos	`VARCHAR2(n CHAR)`; trechos e resposta idempotente em `CLOB`.
Datas	`TIMESTAMP WITH TIME ZONE`; HTTP em UTC ISO 8601.
Exclusão	Sem cascade nas FKs. Aplicação coordena exclusão dos filhos na mesma transação.
Nomenclatura	Tabelas/colunas em português, DTOs HTTP em inglês conforme contrato.

`NOT NULL` consta diretamente no DDL. Campos opcionais estão identificados abaixo. Não persistir percentual ou estado pronto: são derivados dos itens. O mapeamento booleano, identity e timestamp depende dos provedores Oracle escolhidos para JPA/EF e precisa de teste real.

Dicionário — contexto Java

PT_USUARIO

Coluna	Tipo	Regra
ID	NUMBER(18)	PK, identity, positivo.
NOME	VARCHAR2(120 CHAR)	Obrigatório, não só espaços.
EMAIL	VARCHAR2(254 CHAR)	Único, minúsculo e sem espaços externos; normalização e formato validados na API.
SENHA_HASH	VARCHAR2(255 CHAR)	Somente hash seguro produzido pela autenticação Java; nunca senha original.
PAPEL	VARCHAR2(10 CHAR)	CITIZEN ou MANAGER; padrão CITIZEN. Cadastro público não pode escolher papel.
CRIADO_EM	TIMESTAMP WITH TIME ZONE	Padrão SYSTIMESTAMP.

O banco limita os valores do papel; a API restringe quem pode atribuí-los. Há duas identidades sintéticas no dataset de preparativos, mas não são linhas desta tabela nem logins disponíveis.

PT_CATEGORIA

Coluna	Tipo	Regra
ID	NUMBER(18)	PK, identity, positivo.
NOME	VARCHAR2(100 CHAR)	Único, obrigatório e não só espaços.

PT_SERVICO

Coluna	Tipo	Regra
ID	NUMBER(18)	PK, identity, positivo.
CATEGORIA_ID	NUMBER(18)	FK para PT_CATEGORIA.
TITULO	VARCHAR2(160 CHAR)	Obrigatório, não só espaços.
DESCRICAO	VARCHAR2(2000 CHAR)	Descrição para consulta do cidadão.
MUNICIPIO	VARCHAR2(120 CHAR)	Cidade do serviço; não criar entidade Município para o MVP de uma cidade.
VERSAO	NUMBER(18)	Positiva; padrão 1. Alterações relevantes incrementam pela operação central Java.

Índice em CATEGORIA_ID. A FK bloqueia exclusão de categoria que ainda tenha serviços; a API deve mapear o conflito.

PT_REQUISITO

Coluna	Tipo	Regra
ID	NUMBER(18)	PK, identity, positivo.
SERVICO_ID	NUMBER(18)	FK para PT_SERVICO.
TITULO	VARCHAR2(200 CHAR)	Instrução curta, obrigatória e não só espaços.
OBRIGATORIO	NUMBER(1)	0 ou 1; padrão 1.
FONTE_URL	VARCHAR2(2048 CHAR)	Referência de origem; validar formato e fonte na API/processo de revisão.

Índice em SERVICO_ID. O snapshot inclui `id`, `title`, `required` e `sourceUrl`. Não há trigger para aumentar versão automaticamente: manutenção de requisito e incremento usam a mesma transação Java, sem incremento duplicado.

PT_TRECHO_ORIENTACAO

Coluna	Tipo	Regra
ID	NUMBER(18)	PK, identity, positivo.
SERVICO_ID	NUMBER(18)	FK para PT_SERVICO; filtrar a recuperação por serviço.
TEXTO	CLOB	Trecho do material revisado.
FONTE_URL	VARCHAR2(2048 CHAR)	URL da fonte oficial.
FONTE_TITULO	VARCHAR2(200 CHAR)	Título apresentado na referência.
REVISADO_EM	TIMESTAMP WITH TIME ZONE	Data real da revisão.
VERSAO	NUMBER(18)	Positiva; associa o trecho à versão do catálogo.

Índice composto SERVICO_ID/VERSAO. Tabela sem seed nesta base: corpus e recuperação pertencem a Gabriel e precisam de fontes reais escolhidas. O DDL não implementa RAG.

Dicionário — contexto .NET

PT_PREPARATIVO

Coluna	Tipo	Campo HTTP / regra
ID	NUMBER(18)	`id`; PK, identity, positivo.
OWNER_SUBJECT	VARCHAR2(128 CHAR)	`ownerSubject`; vem do JWT validado, nunca do corpo da requisição.

Coluna	Tipo	Campo HTTP / regra
SERVICO_ID	NUMBER(18)	`serviceId`; ID externo positivo, sem FK Java.
VERSAO_ORIGEM	NUMBER(18)	`sourceVersion`; versão positiva capturada na criação.
TITULO	VARCHAR2(160 CHAR)	`title`; obrigatório e não só espaços.
CRIADO_EM	TIMESTAMP WITH TIME ZONE	`createdAt`; padrão SYSTIMESTAMP.

Índices em OWNER_SUBJECT/ID e SERVICO_ID. Toda operação HTTP verifica propriedade, inclusive acesso a itens. Criar cabeçalho, copiar os itens do snapshot e salvar idempotência na mesma transação. Se o Java falhar antes de fornecer snapshot, não persistir preparativo parcial.

PT_ITEM_PREPARATIVO

Coluna	Tipo	Campo HTTP / regra
ID	NUMBER(18)	`id`; PK, identity, positivo.
PREPARATIVO_ID	NUMBER(18)	`preparationId`; FK para PT_PREPARATIVO.
REQUISITO_ID	NUMBER(18), opcional	`requirementId`; ID positivo da origem, sem FK. NULL para item pessoal sem requisito.
TITULO	VARCHAR2(200 CHAR)	`title`; cópia editável da instrução.
OBRIGATORIO	NUMBER(1)	`required`; 0 ou 1, padrão 1.
CONCLUIDO	NUMBER(1)	`completed`; 0 ou 1, padrão 0.

Índice em PREPARATIVO_ID. Não há unicidade por requisito: itens pessoais não têm origem obrigatória, e o contrato não proíbe cópias. A edição do catálogo ou exclusão do requisito não apaga o snapshot pessoal.

Constraints validam os booleanos. O gatilho futuro de Gabriel impedirá mudar PREPARATIVO_ID em item existente. A base ainda não contém esse gatilho.

PT_ITEM_AUDITORIA

Coluna	Tipo	Regra
ID	NUMBER(18)	PK e identity.
ITEM_ID	NUMBER(18)	ID histórico, sem FK.
OPERACAO	VARCHAR2(6 CHAR)	INSERT, UPDATE ou DELETE.
OCORRIDO_EM	TIMESTAMP WITH TIME ZONE	Padrão SYSTIMESTAMP.
USUARIO_BANCO	VARCHAR2(128 CHAR)	Identidade da sessão Oracle; não representa automaticamente o cidadão do JWT.
OLD_PREPARATIVO_ID / NEW_PREPARATIVO_ID	NUMBER(18), opcionais	Vínculo anterior/novo.

Coluna	Tipo	Regra
OLD_REQUISITO_ID / NEW_REQUISITO_ID	NUMBER(18), opcionais	Referência anterior/nova.
OLD_TITULO / NEW_TITULO	VARCHAR2(200 CHAR), opcionais	Instrução anterior/nova.
OLD_OBRIGATORIO / NEW_OBRIGATORIO	NUMBER(1), opcionais	Classificação anterior/nova.
OLD_CONCLUIDO / NEW_CONCLUIDO	NUMBER(1), opcionais	Estado anterior/novo.

Índice em ITEM_ID/OCORRIDO_EM. Em INSERT os campos OLD ficam nulos; em DELETE os campos NEW ficam nulos. O futuro gatilho de Orlando deverá escrever o registro na mesma transação do item. Se houver rollback, o log da alteração também será revertido. Excluir item/preparativo não deve apagar logs já confirmados. Não usar transação autônoma para simular auditoria confirmada de alterações que falharam.

PT_IDEMPOTENCIA

Coluna	Tipo	Regra
OWNER_SUBJECT	VARCHAR2(128 CHAR)	Parte da PK; identidade autenticada.
OPERACAO	VARCHAR2(64 CHAR)	Parte da PK, por exemplo CREATE_PREPARATION.
CHAVE	VARCHAR2(128 CHAR)	Parte da PK; Idempotency-Key, não só espaços.
PAYLOAD_HASH	VARCHAR2(64 CHAR)	SHA-256 hexadecimal minúsculo do payload canônico.
RESPOSTA_STATUS	NUMBER(3)	Status HTTP de sucesso, 200–299.
RESPOSTA_JSON	CLOB	JSON válido para reproduzir o resultado original.
CRIADO_EM	TIMESTAMP WITH TIME ZONE	Padrão SYSTIMESTAMP.

Chave composta evita duplicidade por cidadão/operação/chave. Não há ID adicional nem FK para o preparativo; apagar o recurso não deve tornar uma chave antiga nova. O replay devolve o resultado original registrado.

A API deverá definir serialização canônica do payload, validar o limite da chave, comparar o hash, retornar 409 por conteúdo divergente e tratar corrida pela constraint. A gravação é feita junto ao preparativo/itens, com rollback do conjunto em falha. Este DDL não implementa o fluxo nem comprova concorrência. TTL/expurgo não foi incluído porque não foi definido no contrato.

PT_VERSAO_SERVICO

Coluna	Tipo	Regra
SERVICO_ID	NUMBER(18)	PK; ID externo positivo, sem FK Java.
VERSAO	NUMBER(18)	Última versão conhecida, positiva.

Coluna	Tipo	Regra
ATUALIZADO_EM	TIMESTAMP WITH TIME ZONE	Quando a versão foi registrada; padrão SYSTIMESTAMP.

Tabela técnica prevista no M0. Caso a opção de mensageria seja confirmada, o consumidor aplica o máximo entre versão recebida e conhecida. A comparação com VERSAO_ORIGEM sinaliza fonte desatualizada e não modifica o checklist. O seed fornece versões para testar os contratos; não simula entrega real de evento.

Regras e casos do dataset

`completionPercent = 100 × obrigatórios concluídos / total de obrigatórios`. Sem obrigatórios: percentual 0 e `ready=false`. Opcionais não impedem prontidão. `sourceOutdated` compara versão de origem com a conhecida.

Preparativo	Serviço	Itens	Obrigatórios	Pendentes obrigatórios	%	Pronto	Fonte antiga
101	1	3	2	2	0	não	sim
102	1	3	2	1	50	não	sim
103	1	3	2	0	100	sim	não
104	2	2	2	2	0	não	não
105	2	2	2	0	100	sim	não
106	3	1	0	0	0	não	não
107	3	0	0	0	0	não	não

O caso 103 tem um opcional pendente, mas está pronto. O caso 106 tem somente um opcional concluído e não fica pronto. O caso 107 não tem itens. Esses resultados também constam em `expectedProgress` no manifesto para revisão pelo Gabriel; esse campo não pertence a uma tabela ou documento persistido.

Mapeamento relacional → documental

Representação prevista para a etapa Mongo, ainda sem scripts ou carga executada:

Oracle	MongoDB	Justificativa
PT_SERVICO	`servicos._id`, `title`, `description`, `city`, `version`	Mesmo ID, um documento por serviço.
PT_CATEGORIA	`servicos.category: { id, name }`	Categoria incorporada para leitura por serviço e relatório 1.
PT_REQUISITO	`servicos.requirements[]`	Requisitos incorporados, consultados com o serviço.
PT_PREPARATIVO	`preparativos._id`, `ownerSubject`, `serviceId`, `sourceVersion`, `title`, `createdAt`	Cabeçalho de um preparativo por documento.

Oracle	MongoDB	Justificativa
PT_ITEM_PREPARATIVO	`preparativos.items[]`	Itens pertencem ao preparativo, preservando IDs e snapshot.
PT_VERSAO_SERVICO	Comparação com a versão do serviço no relatório, quando necessária	Não criar terceira collection apenas para copiar tabela técnica.
Usuários, trechos, auditoria e idempotência	Fora da projeção acadêmica	Adisciplina pede representar parte dos mesmos dados. Evita duplicar autenticação/logs sem necessidade.

`preparativos.serviceId` é referência ao documento de serviço; não incorporar o catálogo inteiro em cada preparativo. IDs devem ser BSON Long na projeção, datas BSON Date em UTC, flags booleanas. Carga futura: 3 documentos em `servicos` + 7 em `preparativos`, total 10; requisitos e itens incorporados não contam como documentos de nível superior.

Relatórios previstos: 1) serviços por categoria; 2) preparativos com itens obrigatórios pendentes; 3) agregação por serviço/estado. O relatório 2 deve distinguir ausência de pendência de `ready=true`, por causa dos casos vazio e sem obrigatórios. Gabriel definirá a classificação do relatório 3 de forma igual no Oracle e Mongo, preservando essas exceções.

O que falta validar

- Verificar os mapeamentos das APIs no servidor real; a instalação DDL, carga e testes desta base já passou no Oracle 19c FIAP.
- Mapeamentos de identidade, booleanos, CLOB e timestamp nos provedores efetivos.
- Cidade, três serviços reais, requisitos e fontes oficiais.
- Assinaturas dos packages, gatilhos e testes das regras em Oracle.
- Carga Mongo e comparação efetiva dos três relatórios.
- Fluxos HTTP de autorização, snapshot, idempotência concorrente e atualização de versão.

Não foram criadas integrações automáticas, migrations de API ou infraestrutura.

Validação executada nesta versão

Em 09/11/2026, o conteúdo do script foi executado no Oracle 19c da FIAP por JDBC, em 59 instruções. Antes da instalação não existiam objetos ou constraints reservados ao PaperTrail.

Verificação	Resultado real
DDL e carga	10 tabelas; 2 categorias, 3 serviços, 8 requisitos, 7 preparativos, 14 itens e 3 versões conhecidas.
Constraints e transação	Rejeitados booleanos inválidos, vínculo inexistente, título em branco, versão zero, exclusão com filhos, chave idempotente duplicada e JSON inválido. Rollback restaurou o item.
Identity e referências externas	Geração acima da faixa de fixtures validada; serviço externo não exige FK cruzada.

Verificação	Resultado real
Objetos anteriores	Os nomes das 90 tabelas anteriores permaneceram iguais. O script não executa DML nelas.
Verificação local	IDs, relações, versões, booleanos, cenários de progresso e trecho SQL gerado conferidos pelo gerador de fixtures.

Não foram testados ainda: execução pelo cliente SQL Developer/SQLcl, integração JPA/EF/HTTP, concorrência da API, packages/gatilhos futuros, MongoDB e equivalência dos relatórios. A checagem da chave idempotente prova a constraint de unicidade, não o fluxo HTTP de replay.

A base já está instalada nessa conta. Reexecutar o instalador deve parar na checagem de nomes; para continuar implementando, aplicar somente mudanças novas revisadas. Não remover objetos para contornar essa proteção.

Divisão da implementação

Responsável	Próximos objetos
Orlando	PKG_PREPARATIVO: percentual_concluido e criar_preparativo; gatilho de auditoria; bloco de progresso; Mongo servicos e relatório 1 com SQL equivalente.
Gabriel	PKG_CHECKLIST: quantidade_pendente e alterar_estado_item; gatilho contra transferência de item; bloco de pendências; Mongo preparativos e relatórios 2à3 com SQL equivalente.

Packages sem COMMIT interno. A procedure criar_preparativo cria o cabeçalho; a aplicação inclui os itens na mesma transação. Erros reservados: Orlando à20001..à20019 e Gabriel à20020..à20039. O uso de à20001 na verificação inicial do instalador indica colisão de nomes, antes de qualquer DDL.

Cobertura da disciplina: estado real

Item do edital	Pontos	Estado nesta base
Modelo Oracle e 2 blocos PL/SQL	10	Modelo escrito; blocos demonstrativos pendentes. O bloco de teste não substitui as demonstrações de negócio.
2 funções e 2 procedimentos	15	Pendentes.
2 gatilhos, incluindo auditoria	10	Tabela de log pronta; gatilhos pendentes.
2 packages spec/body e testes	10	Pendentes.
Mongo: 2 collections e 10 documentos	10	Mapeamento e dataset definidos; carga pendente.
Mapa e 3 relatórios, com agregação	15	Mapa definido; consultas e comparação real pendentes.

Parte técnica: 70 pontos. Defesa: 30 pontos, com demonstração de modelo, função/procedure, auditoria/packages, carga Mongo, comparação de relatórios e justificativas. Evidências ainda serão registradas. A meta de nota não à garantia de resultado.

O edital admite README/PDF e exige scripts .sql e .js. Um SQL Único à a organização escolhida pela equipe; a preferência do professor não foi confirmada. MongoDB precisará de script JavaScript próprio, pois PL/SQL não à executado no Mongo.

Entrega, penalidades e defesa

Entrega pelo Portal do Aluno, com SQL Oracle, JavaScript Mongo, PDF e TXT de identificação do grupo. O ZIP aparece no edital como exemplo de formato; confirmar a embalagem final com o professor. Apresentação presencial de 23 a 27/11, na aula da disciplina.

Ocorrência	Consequência no edital
Fora do prazo ou do Portal	−50 pontos
Arquivo corrompido, incompleto ou sem TXT de identificação	Até −20 pontos
Sem relação com CitizenTech	Nota zero
Objetos Oracle ou scripts Mongo não executam	Até −50 pontos
Relatórios vazios ou transição não explicada	Até −30 pontos

Defesa: seis itens de 5 pontos — problema/modelagem Oracle; execução de função/procedure; gatilho/auditoria/packages; criação/carga Mongo; três relatórios comparados; justificativas e respostas. A demonstração precisa usar execução real.

Referências

- Edital local: 2TDS Fevereiro à Global Solution 2026-2, páginas 70à76.
- Planejamento PaperTrail PT-DIV-1: contratos, divisão e roteiro de Orlando.
- Oracle 19c à CREATE TABLE e identities.
- Oracle 19c à JSON em CLOB e validação.

Dados de demonstração fictícios. URLs example.invalid não são fontes oficiais. Os subjects fixture-citizen-a/b não são logins disponíveis. Não utilizar esses requisitos para orientar cidadãos.